

Transformer Model: 50 Q&A

1. Q: What is a token in language models?

A: A token is a unit of text, such as a word or subword, used as input to language models. Tokenization splits text into tokens.

2. Q: What is embedding in the context of Transformers?

A: Embedding is a vector representation of tokens that captures their semantic meaning in a continuous space.

3. Q: What is self-attention?

A: Self-attention is a mechanism that allows each token in a sequence to attend to every other token, enabling contextual understanding.

4. Q: What is the purpose of positional encoding?

A: Positional encoding injects information about the position of tokens in a sequence since the Transformer has no recurrence.

5. Q: Why use Transformers over RNNs?

A: Transformers enable parallelization, handle long-range dependencies better, and scale more efficiently than RNNs.

6. Q: What is multi-head attention?

A: Multi-head attention allows the model to focus on different positions and aspects of the input simultaneously.

7. Q: What is the role of the feed-forward network in Transformer blocks?

A: It processes the output of the attention mechanism and helps capture complex transformations of the data.

8. Q: How is the attention score computed?

A: It is computed using the dot product of the query and key vectors, scaled, and passed through a softmax to get attention weights.

9. Q: What are Q, K, and V in attention?

A: Q (Query), K (Key), and V (Value) are linear transformations of input embeddings used to calculate attention scores and outputs.

10. Q: What is the final output of a Transformer layer?

A: It is a transformed sequence of embeddings after attention and feed-forward processing, used for prediction or passed to the next layer.

11. Q: What is Byte Pair Encoding (BPE)?

A: BPE is a tokenization algorithm that splits text into subwords based on frequency, helping to balance vocabulary size and representation.

12. Q: Why are embeddings important in NLP?

A: Embeddings convert tokens into dense vectors that capture semantic relationships and can be input into neural networks.

13. Q: Why is attention called 'self-attention' in Transformers?

A: Because each token in the sequence attends to other tokens in the same sequence to gather contextual information.

14. Q: What is the benefit of using multiple attention heads?

A: Multiple heads capture different types of relationships and contexts between tokens, improving representation quality.

15. Q: How is attention different from convolution?

A: Attention is dynamic and depends on content similarity, whereas convolution uses fixed local receptive fields.

16. Q: How is positional encoding implemented in Transformers?

A: Positional encodings are added to token embeddings to indicate their position using sine and cosine functions or learned vectors.

17. Q: Why don't Transformers use recurrence or convolution?

A: Because attention alone is sufficient to model dependencies, enabling better parallelization and long-range interaction.

18. Q: What is Layer Normalization?

A: Layer normalization standardizes the inputs across features for each training example, improving training stability.

19. Q: What is the role of the encoder and decoder in Transformers?

A: The encoder processes the input sequence into representations, and the decoder generates the output sequence using those representations.

20. Q: Why do Transformers use residual connections?

A: Residual connections help gradient flow and prevent vanishing gradients, improving training of deep networks.

21. Q: What is fine-tuning in Transformers?

A: Fine-tuning adjusts a pre-trained model on a specific task using additional labeled data.

22. Q: What is instruction fine-tuning?

A: It involves training models on human-annotated prompts and outputs to align model behavior with user intent.

23. Q: How does self-supervised learning differ from supervised learning?

A: Self-supervised learning uses unlabeled data with artificially created labels, while supervised learning uses human-labeled data.

24. Q: What are some common applications of Transformers?

A: Applications include text generation, translation, summarization, question answering, and code generation.

25. Q: What is the benefit of pre-training a Transformer?

A: Pre-training on large corpora enables the model to learn general language patterns, which can be fine-tuned for specific tasks.

26. Q: What are some limitations of Transformers?

A: Transformers can be compute-intensive, require large datasets, and sometimes struggle with reasoning or out-of-distribution generalization.

27. Q: What is masked attention used for?

A: Masked attention prevents the model from attending to future tokens during training, preserving causality in generation.

28. Q: How does the Transformer handle long sequences?

A: It attends to all positions at once, which can be costly; alternatives include sparse or linear attention mechanisms.

29. Q: What is the role of the softmax function in attention?

A: Softmax converts attention scores into probabilities that weight how much focus is given to each token.

30. Q: What are query, key, and value matrices used for?

A: They help compute attention weights (query-key) and combine information (value) for output.

31. Q: What is cross-attention in Transformer decoders?

A: Cross-attention allows the decoder to attend to encoder outputs, integrating source information for generation.

32. Q: What is the difference between encoder-only and decoder-only models?

A: Encoder-only models (e.g., BERT) focus on understanding tasks, while decoder-only models (e.g., GPT) focus on generation.

33. Q: What is pre-norm vs post-norm in Transformer layers?

A: Pre-norm applies layer normalization before attention/FFN, improving training stability compared to post-norm.

34. Q: Why use dropout in Transformers?

A: Dropout regularizes training by randomly zeroing activations, reducing overfitting.

35. Q: What is label smoothing?

A: Label smoothing prevents overconfidence by distributing some probability mass from the true label to others.

36. Q: What are learning rate schedules used for?

A: Learning rate schedules adjust the learning rate over time, often starting high and decaying for convergence.

37. Q: What is warm-up in training Transformers?

A: Warm-up gradually increases the learning rate at the start of training to prevent unstable updates.

38. Q: What is gradient clipping?

A: Gradient clipping limits gradient norms to prevent exploding gradients in deep networks.

39. Q: What is the advantage of mixed precision training?

A: Mixed precision uses float16 and float32 to speed up training and reduce memory while maintaining accuracy.

40. Q: What is zero-shot learning with Transformers?

A: Zero-shot learning uses prompts without fine-tuning, relying on pre-trained knowledge to perform tasks.

41. Q: What are scaling laws in language models?

A: Scaling laws describe how performance improves predictably with model size, data, and computation.

42. Q: What is sparse attention?

A: Sparse attention reduces computation by restricting attention to a subset of tokens, improving efficiency.

43. Q: What is a Transformer adapter?

A: Adapters are lightweight modules added to frozen pre-trained models to enable efficient fine-tuning.

44. Q: What is parameter quantization?

A: Quantization reduces model size by using lower precision (e.g., int8) representations for weights.

45. Q: What is knowledge distillation?

A: Distillation transfers knowledge from a large teacher model to a smaller student model.

46. Q: How can attention weights aid interpretability?

A: Attention weights highlight which tokens the model focuses on, offering insight into decision-making.

47. Q: What are attention head pruning techniques?

A: Pruning removes less important heads to reduce model size with minimal performance loss.

48. Q: What is the computational complexity of self-attention?

A: Self-attention has $O(n^2)$ time and memory complexity in sequence length n .

49. Q: What are memory-efficient Transformer variants?

A: Variants like Reformer, Longformer, and Performer optimize attention for long sequences and lower complexity.

50. Q: What is in-context learning?

A: In-context learning uses examples in the input prompt to guide model behavior without fine-tuning.